

Golden Rule -> retrieve relevant context first, then let the LLM generate an answer grounded in it

1. What Is RAG?

Retrieval-Augmented Generation (RAG) means: before the LLM answers, the system retrieves the most relevant pieces of text from an external knowledge source and feeds them to the model as context. The model then generates its answer grounded in that retrieved text, instead of relying only on what it memorized during training.

Think of it as an open-book exam: a plain LLM answers from memory alone; a RAG system is handed the right page of the textbook first, then asked the question.

2. Why Direct Prompting Falls Short

Problem	What It Looks Like	Why RAG Helps
Hallucination	Model states wrong facts confidently and fluently	Answer is grounded in real, retrieved source text
Knowledge cutoff	Model has no idea about recent events or new documents	Retrieval pulls current documents at query time, not training time
No citations	You can't tell where an answer came from	Retrieved chunks can be shown/cited directly to the user
Private / internal data	Model never saw your company's documents	You supply your own documents as the knowledge base

3. The RAG Pipeline

Every RAG system follows the same six stages, in order:

```
$ Load -> read raw documents (PDF, text, web pages, etc.)
$ Split -> break documents into smaller chunks
$ Embed -> convert each chunk into a vector
$ Store -> save vectors in a vector database
$ Retrieve -> find the top-k chunks most similar to the query
$ Generate -> LLM answers using the query + retrieved chunks
```

4. RAG vs Fine-Tuning vs Direct Prompting

Approach	Best For	Limitation
Direct prompting	Quick answers from general knowledge	No access to your data; hallucination risk on facts
Fine-tuning	Teaching a model a style, format, or behaviour	Costly and slow to update; not a good way to add fresh facts
RAG	Answering questions from your own / current documents	Answer quality depends entirely on retrieval and chunking quality

5. When To Reach For RAG

- Answers must be grounded in specific documents (policies, manuals, contracts).

Golden Rule -> retrieve relevant context first, then let the LLM generate an answer grounded in it

- The knowledge base changes often and retraining a model isn't practical.
- You need traceability — the ability to cite where an answer came from.
- Users ask about things outside the model's training cutoff.

Pitfall

RAG does not fix a weak LLM's reasoning, and it does not fix bad chunking. Garbage retrieval in means a garbage-grounded answer out — the LLM can only be as good as the context it was handed.

Remember

RAG augments the prompt with retrieved context; it does not retrain or fine-tune the model. The LLM's weights never change — only what it sees at inference time.

INTERVIEW PREP

Q: What problem does RAG solve that prompting alone can't?

A: It grounds answers in an external, up-to-date knowledge source instead of relying solely on what the model memorized during training, which reduces hallucination and fixes the knowledge-cutoff problem.

Q: How is RAG different from fine-tuning?

A: Fine-tuning changes the model's weights to shape style or behaviour and is costly to update; RAG leaves the model unchanged and retrieves fresh context at query time, so updating knowledge just means updating the document store.

Q: Name the six stages of a RAG pipeline.

A: Load -> Split (chunk) -> Embed -> Store (vector DB) -> Retrieve -> Generate.

Q: Does RAG guarantee the model won't hallucinate?

A: No. It reduces hallucination by grounding the answer, but a weak retriever, poor chunking, or the model ignoring its context can still produce a wrong answer.

Q: When would you combine RAG with fine-tuning?

A: When you need both up-to-date factual grounding (RAG) and a specific tone, format, or behaviour baked into the model (fine-tuning) — e.g. a support bot fine-tuned on your brand voice that also retrieves current policy documents.